

Watchtower

Ethereum cannot see its own block.
Creditcoin **can.**

A protocol for insurance that pays out on cryptographic proof of an Ethereum transaction, verified inside a Creditcoin contract by the Attestcoin Protocol's Block Prover Precompile, priced against a proven Chainlink answer, and settled in a single block. No claims adjuster, no oracle operator, no multisig, no trusted watcher.

VERSION	DATE	SETTLEMENT
1.0	SEPTEMBER 2026	CREDITCOIN CC3 TESTNET (102031)
SOURCE CHAINS	LICENCE	
ETHEREUM MAINNET + SEPOLIA	MIT	

Two things happen on Ethereum every day that Ethereum itself cannot act on. A searcher brackets a victim's swap at consecutive positions inside one block, and no contract on that chain can read the fact, because a contract cannot observe the transactions beside it in its own block. A custodian mints more than it locked, and no contract on that chain can audit a whole history inside a single transaction. The evidence for both is sitting in the ledger. It is simply unreachable from inside the chain that produced it.

The industry's answer to both has been to trust somebody: an operator, a committee, a watcher, an insurance desk with a claims process. That trusted party is the single point of failure, and it is the thing the Attestcoin Protocol removes. Creditcoin ships a native Block Prover Precompile at address 0x0FD2 that will verify any Ethereum transaction against an attested block, and it exposes `calculateTxIndex`, which returns that transaction's position inside its block. Those two facts together mean a Creditcoin contract can prove statements about Ethereum's execution order, synchronously, on-chain, for a fraction of a cent.

Watchtower is what that capability is worth in a product. It is an insurance engine in which the claim *is* the proof. A policyholder never files anything. A verified window of Ethereum transactions is handed to a pure rule, the rule returns a verdict, and the vault pays inside the same transaction that proved the harm. Four rules are deployed today: intra-block extraction, reserve conservation, a proven Chainlink price import, and failed-transaction gas reimbursement. A fifth risk means writing one pure function; the engine, the vault, the bounty schedule and the fraud-proof machinery come for free.

This paper describes the Creditcoin infrastructure the system is built on, the protocol design it made possible, the economic design that makes prosecution a market rather than an architectural claim, the security model and its bounds, and measured results from a live deployment. A real Ethereum mainnet sandwich has been prosecuted on Creditcoin CC3 Testnet: three transactions verified in one batch call, damages priced at 3.77 US dollars from the ETH/USD answer Chainlink itself published, settled for 1.64 percent of one Creditcoin block in gas. The paper ends with the things Watchtower has not proven, stated here rather than left to be discovered.

0x0FD2	102031	1.64%	79
BLOCK PROVER	CHAIN ID	OF ONE BLOCK	TESTS
Native precompile. Verifies any Ethereum transaction.	Creditcoin CC3 Testnet. Where every verdict lives.	Cost of a complete three-transaction prosecution.	Offline, plus fuzzed invariants and a real-bundle replay.

01	The problem	04
	Two failures Ethereum cannot act on · why the source chain cannot help itself · the cost of the trusted-party answer · contribution	
02	Creditcoin as verification infrastructure	06
	The Attestcoin Protocol · attestation and continuity proofs · the Block Prover Precompile · calculateTxIndex · batch verification · ChainInfo · the Proof Builder · EvmV1Decoder · what the protocol does not provide · why Creditcoin rather than a bridge, an oracle or a light client · writability	
03	Protocol design	12
	Four planes · subjects · rules · evidence windows · the single entryptpoint · the rule-scoped replay guard · accumulators · settlement modes · gap challenges	
04	The rule library	18
	IntraBlockExtraction · ReserveConservation · ChainlinkFeed · FailedTx · GapChallenge · writing a fifth rule	
05	Economic design	21
	Cover · tranches and risk isolation · three bounds on a payout · premiums · bounties derived from the protocol's own cost curve · bonds and slashing · agent analysis	
06	Security model	24
	Trust boundaries · adversaries and their bounds · guarantees held up by tests · verifying the Creditcoin integration itself	
07	Implementation	27
	Repository · contracts · the prosecutor worker · the dashboard · engineering notes Creditcoin forced	
08	Measured results	30
	Gas from receipts, not estimates · a real mainnet sandwich prosecuted · what actually costs the gas	
09	Limits	32
	What Watchtower has not proven, stated rather than discovered	
10	Roadmap	34
	Writability and source-chain enforcement · permissionless rules · pro-rata settlement · more source chains	
11	What this means for Creditcoin	35
	A settlement chain that adjudicates other chains, and the demand that creates	
12	Conclusion	37
A–D	Appendices	38
	Addresses and networks · interfaces and data structures · rule identifiers and parameters · glossary	

The problem

Ethereum records everything and can act on almost none of it. The ledger is a complete account of what happened, and a contract executing inside that ledger is given a deliberately narrow view of it: the current state, the current call, and a handful of block-level scalars. Everything else, including the shape of the very block the contract is executing in, is outside its reach.

1.1 Two failures Ethereum cannot act on

The two risks Watchtower prosecutes today are chosen because they share a shape. Both are completely evident in the ledger. Both are unreachable from inside it.

Intra-block extraction

A searcher places a buy immediately before a victim's swap and a sell immediately after it, at consecutive transaction indices in one block, on one pool. The victim receives a worse execution price and the difference is the searcher's. The proof is three transactions and their positions. No Ethereum contract can read those positions, because a contract cannot see the transactions beside it in its own block. The pool contract that executed all three swaps has no way to know it executed all three swaps.

Custodian insolvency

A bridge, a wrapped-asset issuer or a real-world-asset custodian mints more claims than it has locked collateral. Any indexer watching the event log can see it within seconds. Nobody can enforce it, because establishing the fact requires auditing an entire history, and auditing an entire history inside one transaction is not something an Ethereum contract can do either.

These are not exotic edge cases. Intra-block extraction is a permanent, industrial-scale transfer of value from ordinary traders to searchers and builders, and it is the most widely understood unpriced risk in decentralised finance. Custodial shortfall is the failure mode behind a substantial share of the largest losses the industry has suffered. Both are provable. Neither is insurable by anything that lives on the chain where they occur.

1.2 Why the source chain cannot help itself

The limitation is structural rather than incidental. The Ethereum Virtual Machine gives a contract no opcode for the transaction index of the call it is inside, no access to sibling transactions, no access to receipts, and no access to historical state beyond the two hundred and fifty six most recent block hashes. A contract can be told about its neighbours only by a party willing to assert the fact, and an assertion is exactly what a proof system is supposed to replace.

The consequence is that every product built around these risks today is built around a trusted observer. Protective services rely on a relay operator. Solvency dashboards rely on an indexer. Cover protocols rely on a claims committee with discretion over whether an event occurred. In each case the security of the product

collapses to the honesty and the availability of that observer, and the observer is compensated for being trusted rather than for being right.

Today's answer to both failures is to trust someone. That is the single point of failure the Attestcoin Protocol deletes.

1.3 What a proof-native answer requires

To replace the observer, an adjudicating contract needs four things about a foreign transaction, and it needs all four inside a single synchronous call, because anything it cannot establish itself it must once again take on trust.

- 01 Proof that the transaction was included in a canonical block of the source chain.
- 02 Proof of where the transaction sat inside that block, since ordering is the fact at issue.
- 03 Access to the transaction's decoded fields and its receipt, since the harm lives in the logs.
- 04 A way to verify several transactions together under one proof, since a claim about ordering is never a claim about one transaction.

Ethereum can supply none of these to a contract executing on Ethereum. A general-purpose bridge supplies the first, slowly and with its own trust assumptions. A zero-knowledge light client can in principle supply the first and the third at considerable cost and latency. The second and the fourth, taken together, are the interesting part, and they are what Creditcoin supplies natively.

1.4 Contribution

This paper makes four claims and supports each with a running implementation.

- **A coordinate is enough.** The triple of source chain, block height and transaction index, delivered by the protocol rather than asserted by a submitter, is sufficient to adjudicate an entire family of cross-chain risks that no source-chain contract can reach.
- **Insurance is the natural product for it.** When the claim is the proof, the claims process disappears. A policyholder never files anything, an adjuster never exercises discretion, and the payout is a consequence of verification rather than of a decision.
- **The engine generalises and the rules are a catalogue.** One entrypoint, one verification path and one vault serve a sandwich prosecution, a solvency ingestion, a price import, a gas refund and a fraud proof. Adding a risk is writing a pure function.
- **Honesty is enforceable without trust.** Claims whose soundness is self-contained settle instantly. Claims that depend on completeness, which no chain can verify, are bonded and challengeable, and the fraud proof runs through the same entrypoint as ordinary evidence.

Creditcoin as verification infrastructure

Watchtower is not a product that happens to be deployed on Creditcoin. It is a product that cannot be built anywhere else, and this section is the reason why. Everything that follows in this paper depends on the surfaces described here.

2.1 Why adjudication belongs on a different chain

A chain cannot cheaply prove things about itself to itself, because the facts a contract wants are precisely the ones the execution environment withholds. It can, however, prove things about itself to somebody else, because a block is a commitment and a Merkle root is a compact, checkable summary of the transactions underneath it.

That asymmetry is the whole opportunity. A second chain that can validate Ethereum's commitments has strictly more visibility into an Ethereum block than an Ethereum contract does. It can read a transaction's neighbours. It can audit a run of history. It can do both inside one transaction of its own and act on the result in the same breath, which is what turns forensic commentary into a settlement.

A Creditcoin contract can see what an Ethereum contract cannot: its neighbours.

2.2 The Attestcoin Protocol

Creditcoin's Attestcoin Protocol turns that asymmetry into a first-class runtime feature. Rather than shipping a verification library that contracts must deploy and maintain, or a bridge that contracts must ask nicely, the protocol places the verifier in the node itself as a native precompile, callable by any contract on the chain, and feeds it a continuously attested view of supported source chains.

Three properties of that design matter more than any individual function signature.

Verification is native, not contractual

The Block Prover is Rust in the Creditcoin runtime, not EVM bytecode. Merkle verification, a workload that is expensive in the EVM and trivial in native code, executes at native speed and is billed at a flat, predictable rate. This is the difference between a proof costing a fraction of a Creditcoin block and a proof being economically absurd.

Verification is unbound

Nothing ties a verified transaction to the contract asking about it. A Creditcoin contract can verify a transaction belonging to a protocol it has no relationship with, sent by a party it has never interacted with, in a block it did not anticipate. This is what allows Watchtower to adjudicate other people's protocols on behalf of other people's users.

Verification is synchronous

The result is available inside the calling transaction. There is no callback, no request identifier, no second transaction to settle against a cached answer. Proof, judgement and payment occupy one block, which is what removes the window an adversary would otherwise have between a proof landing and money moving.

2.3 Attestation and continuity proofs

Underneath the precompile is an attestation pipeline. Attestors observe the source chain and commit its block digests to Creditcoin, so the runtime holds an authoritative, incrementally extended record of Ethereum's history. Verification then reduces to two questions: does the supplied Merkle proof reconstruct the transaction root of some block, and is that block genuinely part of the attested chain.

The second question is answered by a **continuity proof**: a chain of block digests linking the block in question to an attested endpoint the runtime already trusts. The number of roots in that chain is the distance from the evidence to the nearest attestation, and it is the dominant cost of a submission. Recent evidence sits roughly ten hashes from a live attestation. Evidence older than the twenty-four-hour checkpoint boundary needs roughly a thousand, and costs about ten times as much gas.

WHY THIS MATTERS TWICE

Continuity length is not only a cost. It is a measurement of freshness produced by the protocol itself, with no clock, no oracle and nothing a submitter can influence. Section 5.5 shows how Watchtower's entire prosecutor incentive schedule is derived from this one quantity, which means the incentive curve and the chain's real cost curve can never drift apart.

Attestation is not instantaneous. End to end, a fresh Ethereum block becomes provable on Creditcoin in roughly eight minutes, which is materially longer than the attestation cadence alone would suggest. Watchtower treats this as a first-class property of the system rather than an inconvenience: the worker waits explicitly with the protocol's documented polling parameters, the interface narrates the wait rather than hiding it behind a spinner, and the product is designed so that nothing time-critical depends on sub-minute proof latency.

2.4 The Block Prover Precompile at 0x0FD2

The Block Prover Precompile is the only protocol surface Watchtower's settlement contract talks to, and every verdict in the system passes through it. Four of its overloads are used.

SURFACE	USED FOR	CALL SITE
verifyAndEmit (batch)	Every multi-transaction window. One call, one continuity proof shared by the whole window, a state-changing verification that emits an on-chain record.	WatchtowerCore._verifyAndBuild
verifyAndEmit (single)	Single-transaction windows: failed-transaction claims and gap challenges.	WatchtowerCore._verifyAndBuild
verify (read-only)	Dry runs. The dashboard shows a verdict before anyone pays for it, and the worker runs the same check as pre-flight. Costs no gas.	previewEvidence, preflight.ts
calculateTxIndex	The position of a transaction inside its block, derived from its Merkle proof by the protocol.	WatchtowerCore._buildWindow

The precompile also emits TransactionVerified on its own account. Watchtower does not emit it and cannot suppress it, which means every verdict the system issues is corroborated on the block explorer by an event the chain itself wrote. That is free third-party evidence for a claim, and it is the sort of thing a protocol-level verifier can offer that a library cannot.

A TRAP WORTH NAMING

The precompiles are native runtime code, so extcodesize at 0x0FD2 returns zero on the chain that has them. A code-length guard, the reflex any Solidity developer reaches for, reports the precompile missing precisely where it exists. The correct guard is NativeQueryVerifierLib.hasPrecompile(), and the same property is why a forked development node does not have the precompile and unit tests must substitute a stateless mock at that address.

2.5 calculateTxIndex, the coordinate the system turns on

A Merkle proof of inclusion in a transactions trie carries more information than the fact of inclusion. The sibling path encodes the leaf's position, and in an Ethereum transactions trie the leaf key is the transaction index. The protocol exposes this directly: calculateTxIndex takes a Merkle proof and returns the index.

Watchtower does not reimplement it. No sibling-path arithmetic was written anywhere in the repository. Every transaction's position is derived by the precompile, inside the settling transaction, after verification and before any ordering is enforced. This is both the project's thesis and its security property: a dishonest claim about ordering cannot survive submission, because the contract never believes the ordering it was handed. It re-derives it.



The adjacency assertion in the middle of that listing is the fact no source-chain contract can check. Everything above it is the protocol's work; everything below it is Watchtower's.

2.6 Batch verification

A claim about execution order is never a claim about one transaction, so a verifier that only handles one transaction at a time would force a protocol to accumulate partial results across several transactions and defend the gaps between them. The Attestcoin Protocol exposes batch verification directly to Solidity: up to ten transactions, spanning up to one thousand blocks, verified in a single call under one shared continuity proof.

This shapes Watchtower more than any other protocol property. It is why an evidence window is a first-class object rather than a sequence of submissions. It is why a three-transaction sandwich prosecution is one transaction with one continuity proof rather than three separate proofs. It is why the stream scanner chunks its work to exactly ten transactions per submission, and it is why the SDK's proof-merging helper has no call site in this system: merging pays off when several chunks combine into one submission, and the batch ceiling means they cannot.

2.7 The ChainInfo Precompile at 0x0FD3

The second precompile reports which source chains the protocol supports and the latest attested height for each. Watchtower uses it for two things. At boot, the worker asserts the mapping from protocol chain key to source chain and refuses to start if it disagrees with its configuration, so a misconfiguration becomes a readable boot failure rather than a confusing revert twenty minutes into a run. At runtime, the attested head is surfaced in the interface, so a reader can see how far the protocol's view of Ethereum has advanced.

Chain keys are a Creditcoin-internal identifier and deliberately not EVM chain ids. Ethereum Sepolia is chain key 1 and Ethereum Mainnet is chain key 3. Mainnet being readable from a testnet deployment is what allows Watchtower to prosecute real, historical Ethereum sandwiches from a testnet contract, which is a considerably stronger demonstration than a staged incident on a test network.

2.8 The Proof Builder service

Proofs are constructed off-chain by the protocol's Proof Builder service, reached through the official SDK. It supplies the three operations the evidence pipeline needs: `waitUntilHeightAttested`, which blocks until a source-chain height is provable; `getProof`, for a single transaction; and `getBatchProof`, which returns Merkle proofs for a set of transactions together with the shared continuity proof that binds them.

The service is untrusted in exactly the way the rest of the off-chain pipeline is untrusted. It produces proofs; the precompile decides whether they are valid. A Proof Builder that returned garbage would cause a revert, not a false verdict.

2.9 EvmV1Decoder

Verification establishes that a byte string is a genuine Ethereum transaction and receipt. Acting on it requires reading the fields inside, and the protocol ships `EvmV1Decoder` for exactly that: sender and recipient, receipt

status, gas used, type-specific gas price, the log array, and filtering of logs by event signature. It is an internal Solidity library, so it inlines into the calling contract, there is no library linking step, and the deployed address of a decoder is irrelevant.

Watchtower uses it in the core to decode sender, recipient and receipt status for every verified transaction, and in every rule to read the logs that carry the harm: Uniswap V2 swap amounts, a custodian's lock and mint events, and a Chainlink aggregator's published answer.

2.10 What the protocol deliberately does not provide

A verification layer is defined as much by what it declines to assert as by what it proves. Five of the protocol's boundaries shaped Watchtower's design directly, and naming them is the fastest way to explain why the system looks the way it does.

No state proofs

The protocol proves transactions and receipts, not account state. There is no way to read a custodian's balance at a height. Watchtower therefore never reads a balance: solvency is replayed forward from an anchor as a running accumulator over proven events. This single constraint is the origin of the cursor, the ordered stream window, the optimistic settlement path and the gap challenge. Almost half the design in Section 3 exists because of it.

Success is not checked for you

Verification establishes inclusion, not outcome. A reverted transaction is just as provable as a successful one. Every path in Watchtower therefore asserts receipt status explicitly, and the one rule that wants the inverse asserts it for itself, so the inversion lives in a rule rather than in the engine.

Batch order is not guaranteed

`getBatchProof` returns nested maps, and map iteration order carries no promise. An unsorted batch is an ordinary occurrence rather than an attack, and it is handled the way both cases must be handled: the worker sorts before submitting, and the contract sorts again after re-deriving every index from the precompile.

Gas estimation does not work against precompiles

The EVM pallet does not reliably propagate revert reasons in estimation mode, so a naive `estimateGas` call is unusable. Submissions use the SDK's `computeGasLimit`, which falls back to a formula driven by continuity length. This is a case where the protocol's own helper encodes knowledge an integrator would otherwise have to rediscover expensively.

Windows are bounded

Ten transactions, one thousand blocks. Watchtower enforces both in the contract rather than relying on the precompile to refuse, and the scanners chunk their work to match.

2.11 Why Creditcoin rather than the alternatives

APPROACH	POSITION INSIDE A BLOCK	SYNCHRONOUS	COST AND LATENCY	TRUST
Ethereum contract, unaided	Unavailable	Yes	n/a	n/a
Oracle or relay operator	Asserted, not proven	No	Low cost, operator latency	The operator
General-purpose bridge	Not exposed	No	High latency	Bridge validator set
Zero-knowledge light client	Possible, at cost	Rarely	High proving cost and latency	Circuit and setup
Attestcoin Protocol on Creditcoin	Native, via <code>calculateTxIndex</code>	Yes	Roughly 1.6 percent of a block for three transactions	Creditcoin consensus and attestors

The comparison is not that the alternatives are bad. It is that none of them expose the one fact this product needs. An oracle could be asked to report a transaction index, but a reported index is an assertion by an operator and the entire point is to remove that operator. A bridge proves inclusion without exposing position. A light client could in principle prove position, at a proving cost and latency that would make a three-dollar insurance claim economically impossible. Creditcoin exposes position natively, synchronously, and for a fraction of a cent, and it is the only chain that does.

2.12 Writability, and what it will change

Everything described so far is *readability*. Creditcoin proving facts about another chain. The protocol's outbound direction, *writability*, is in third-party audit at the time of writing, and Watchtower deliberately keeps it off the critical path. The system is architected for it and demonstrates without it.

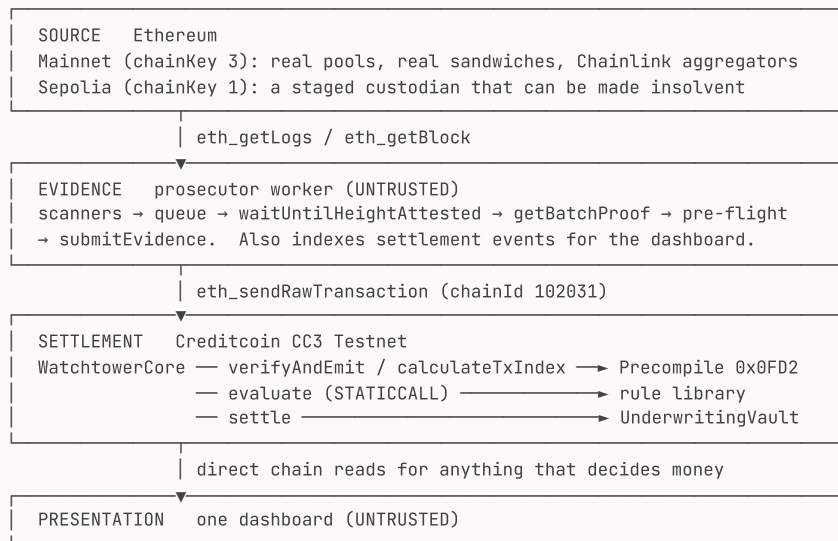
The path is concrete rather than aspirational, because the outbound suite already ships in the protocol's contract package: an outbox on the sending side, an attestor registry, a relayer, a proof verifier and an inbox on the destination side. A settled Watchtower verdict publishes a message to the destination chain's outbox, attestors sign it, a relayer delivers it, and restitution is enforced at the source rather than reimbursed on Creditcoin.

Readability proves the harm. Writability undoes it.

Protocol design

Watchtower is four layers with one direction of travel. Evidence is found on Ethereum, assembled off-chain by an untrusted worker, verified and settled on Creditcoin, and read back by a browser that trusts the chain over the worker. Three of those four components are deliberately not trusted, and the one that is trusted is a contract whose behaviour was fixed before any incident occurred.

3.1 Four planes



3.2 Subjects

Everything in Watchtower attaches to a **subject**: one contract on one source chain, bound to one rule that says what can be proven about it. Cover, capital and bounties all attach to subjects, and every incident names one. A subject is a risk the system has agreed to have an opinion about, together with the exact rule by which it will be judged.

FIELD	MEANING
chainKey	Which source chain. The worker asserts this mapping against the ChainInfo precompile at boot.
sourceContract	The address being watched: a pool, a bridge, a Chainlink aggregator, an account.
boundRule	The one rule allowed to judge it. Evidence filed under any other rule is rejected before a proof is looked at.
kind	Pool, Custodian, Feed or Account. Presentation and scanner routing only; the rule decides everything.
anchorHeight, anchorIndex	The coordinate a stream rule begins accumulating from.
payoutCapPerBlock	The ceiling on what one block of evidence can cost the vault. This is the bound on a wrong judgement by an untrusted rule.
priceSubject	A pointer to the feed subject whose proven answer denominates this subject's damages.
state	The accumulator a stream rule maintains: locked, minted, price, and the proven cursor.

Subject identifiers are derived rather than assigned. The identifier is the hash of chain key, source contract and rule identifier, which has two useful consequences. The same contract judged by the same rule is always the same subject and cannot be registered twice. And the same address can legitimately appear as several subjects under different rules, so a bridge can be watched for solvency and for failed transactions at once, as two separate risks with separate capital behind them.

```
subjectId = keccak256(abi.encode(chainKey, sourceContract, ruleId))
```

Registration is owner-gated in version one. That gate is a policy choice rather than a security requirement: rules are pure and payouts are capped, which is exactly what makes permissionless subject and rule submission safe later. The security model does not change when the gate opens, only the gate.

3.3 Rules

A rule answers one question about a verified window of Ethereum transactions: did this happen, who was harmed, and by how much. It declares in advance the shape of window it is willing to inspect and the settlement mode its claims require, and it implements `evaluate` as a view function of the window and the subject's accumulator.

Because `evaluate` is a view function, the core reaches it by `STATICCALL`. A rule therefore cannot write state and cannot move money. The worst thing a hostile or simply incorrect rule can do is return a wrong number, and that number is bounded by the subject's per-block payout cap. This is what makes an open rule library a manageable economic problem rather than an unbounded one.

The test suite deploys a hostile rule that matches the expected selectors but declares `evaluate` state-mutating. The call reverts. The guarantee is a passing test rather than a paragraph in a document.

3.4 Evidence and window shapes

Evidence is a **window**: one or more Ethereum transactions, each with a Merkle proof, sharing one continuity proof, submitted together and verified in a single call to the precompile. The window is what makes a claim about execution order possible, because such a claim is never about one transaction. It is about several, and their positions.

SHAPE	CONSTRAINT	USED BY
SINGLE_TX	Exactly one transaction. No ordering to check.	FailedTx, GapChallenge
INTRA_BLOCK_ADJACENT	All in one block, at strictly consecutive indices.	IntraBlockExtraction
SEQUENTIAL_STREAM	Strictly ascending by height and index, starting beyond the proven cursor.	ReserveConservation, ChainlinkFeed

The shape is declared by the rule and enforced by the core before `evaluate` is ever called, so a rule never has to defend itself against a window it did not ask for. Ordering is enforced against indices the contract derived from the precompile, never against indices a submitter supplied.

3.5 submitEvidence, the single endpoint

One function handles every risk in the system. A sandwich prosecution, a reserve ingestion, a price update, a gas-refund claim and a fraud proof are the same call with different rule identifiers. This is not a convenience: it is what guarantees that the fraud proof defending the system runs through exactly the same verification path as the evidence it disputes.

```
function submitEvidence(EvidenceInput calldata input)
    external
    payable
    returns (bytes32 incidentId);
```

- 01 Load the subject and the rule. Reject a rule not bound to this subject, a paused subject, or a chain key the subject does not watch.
- 02 Verify the whole window through the precompile, using the batch overload when it holds more than one transaction.
- 03 Derive every transaction index from `calculateTxIndex`.
- 04 Decode sender, recipient and receipt status with `EvmV1Decoder`.
- 05 Sort by block height and index, then apply the rule-scoped replay guard.
- 06 Enforce the window shape the rule declared.
- 07 Call `rule.evaluate` by `STATICCALL`.

- 08 Apply the accumulator delta the rule asked for.
- 09 Settle instantly, or open a bonded claim with a challenge window.
- 10 Emit the acceptance record and either a verdict or a breach.

The order carries the argument. Indices are derived after verification and before sorting, so the ordering the contract enforces is the protocol's ordering rather than the submitter's claim about it. The replay guard runs before the shape check, so a duplicate is rejected cheaply rather than after a rule has been consulted. And `evaluate` is the last thing that happens before money moves, which is precisely why it is a static call against a view function.

A companion function, `previewEvidence`, runs the same path against the precompile's read-only overload. It costs no gas, changes no state, and lets the interface show a verdict before anyone pays for one. The worker uses the same surface as its pre-flight, so an unprovable window is rejected before gas is spent on it.

REVERT	MEANING
Rule not bound to this subject	Evidence filed under the wrong rule.
Subject inactive	The subject was retired. Verdicts already issued against it still stand.
Chain key mismatch	Evidence from a chain this subject does not watch.
Malformed input	More than ten transactions, or mismatched proof arrays.
AlreadyConsumed	This exact rule, subject, chain, height and index combination was already used.
Shape violation	The window is not what the rule's declared shape requires.
VerificationFailed	The precompile refused. The evidence was not real.
UnsupportedTxType	A blob or delegation transaction reached a rule that cannot decode it.

A REVERT IS THE SYSTEM WORKING

Every one of these checks happens inside the settling transaction, which is what makes the worker safe to leave untrusted. Bad evidence costs its submitter gas and pays nobody.

3.6 The replay guard is rule-scoped

The natural replay guard for a verification contract is a global one keyed on chain, height and index. Watchtower cannot use it, because one Ethereum transaction is legitimately evidence under several rules at once. A custodian's lock transaction feeds the reserve accumulator, and the same transaction may later prove that the accumulator skipped it.

```
key = keccak256(ruleId, subjectId, chainKey, blockHeight, txIndex)
```

A test named for this case fails outright under a global guard. The rule-scoped key is not a refinement; without it the fraud proof described in Section 3.9 cannot be filed at all.

3.7 Accumulators and constant-size storage

Because the protocol proves transactions rather than account state, every balance in this system is replayed rather than read. A stream rule maintains an accumulator per subject: locked, minted, the last proven price, and a cursor giving the coordinate of the last applied entry. Each submission must start strictly beyond that cursor, which is what makes the replay ordered and inspectable.

Horizontal evidence, by contrast, is verified, judged and discarded inside one transaction, and only a verdict record remains. The result is that storage is constant no matter how much Ethereum history has been ingested, which is what stops an insurance protocol from turning into an archive nobody can afford to write to.

3.8 Settlement: instant and optimistic

Not every claim has the same epistemic standing, and the protocol says so in its settlement modes rather than pretending otherwise.

	SANDWICH	SOLVENCY
Soundness	Self-contained. The three proofs are the entire claim.	Depends on the completeness of a stream.
Settlement	INSTANT	OPTIMISTIC, bonded, with a challenge window
Defence	None needed	Gap challenge: prove a skipped transaction inside the claimed range

You cannot prove a negative on-chain, so Watchtower does not pretend to. The asymmetry is a design feature, stated rather than hidden.

Three adjacent indices in one block either are what the sandwich rule describes or they are not, and the precompile settles that question inside the transaction. A solvency claim depends on the prosecutor having skipped nothing, and no on-chain check can establish an absence. Those claims are therefore bonded and left open for a window: economically secured rather than self-evident.

3.9 Gap challenges

An optimistic claim asserts something no chain can check: that the prosecutor ingested every relevant transaction in the range they claimed to cover. Rather than assume it, Watchtower prices it. The prosecutor

escrows a bond, the claim sits open for a challenge window, and anyone who can show a skipped transaction takes the bond.

- 01 Find a transaction touching that subject which falls strictly inside the range the claim covered and was not ingested.
- 02 File it as an ordinary single-transaction submission through the same entryptpoint, against the open incident.
- 03 The core verifies it exactly as it verifies any other evidence: the precompile, the index derivation, the decoder, all of it.
- 04 It then checks that the transaction is strictly inside the claimed range, and that at least one of its logs was emitted by the subject's own source contract.
- 05 The accumulator rolls back to the snapshot taken before the disputed ingestion, and the prosecutor's bond becomes the challenger's.

The fraud proof is an ordinary verified window. Watchtower's defence runs on Watchtower.

Two details are worth drawing out. The relevance check means a challenger cannot roll back a claim with an unrelated transaction that merely happens to fall in the right range. And open breaches are counted rather than flagged: a stream can break twice before anyone settles the first claim, and a boolean would have thawed the underwriters' tranche after the first payout while the second was still in dispute.

Once a challenge window closes unchallenged, anyone may settle the breach. It is a permissionless call, so a claim cannot be left in limbo by a prosecutor who lost interest.

The rule library

Four rules are deployed. Three of them price risk and one of them imports the price the other three are denominated in. Together they are about four hundred and sixty lines of Solidity, which is the point: the engine is the product and the rules are a catalogue.

RULE	WINDOW	SETTLEMENT	BENEFICIARY
IntraBlockExtraction	3 adjacent, one block	Instant	Named by the rule
ReserveConservation	Ordered stream, at most 10	Optimistic	The subject's cover holder
ChainlinkFeed	Ordered stream	Source rule	Nobody. It imports a price.
FailedTx	1 transaction	Instant	Named by the rule

4.1 IntraBlockExtraction

This is the rule that cannot exist on Ethereum. It requires three transactions at strictly consecutive indices inside one block, all touching the same pool, with the same sender on the outside and a different party in the middle, and the outer pair completing a round trip in profit in the same direction the victim traded.

Every one of those conditions is checkable from the verified window, and the one that makes the claim possible, adjacency, comes from the protocol. A contract on the source chain cannot reach it. What was previously forensic commentary becomes an on-chain verdict with money attached.

The damages model

Damages are stated plainly so they can be argued with. A sandwich is a round trip: the searcher spends one token to buy another immediately before the victim, then sells it back immediately after. Whatever extra they end up with came out of the victim's execution price, because inside one block there is no other source.

```
extracted = (input token received in the back-run) - (input token spent in the front-run)
```

This is a lower bound rather than a reconstruction of the victim's counterfactual price. It ignores the pool fee the attacker paid, so it understates rather than overstates the loss, and it needs only the two swap logs, which are themselves proven. There is no reserve snapshot, no simulation, and nothing a prosecutor could fabricate. The verdict is proof; the pricing is a documented policy, and the distinction is maintained everywhere in the system.

The dollar figure attached to it is itself proven. The extracted amount is converted at the ETH/USD answer Chainlink published on Ethereum, imported by the feed rule described in Section 4.3 from that aggregator's own transaction. Proven, not reported.

IT DECLINES RATHER THAN GUESSES

A pool has two tokens and a linked feed prices exactly one of them. The deployed instance is configured for the eighteen-decimal side of its pair. A sandwich the searcher funded with the six-decimal side is real, but this instance cannot price it, and it returns no verdict rather than publishing a figure wrong by twelve orders of magnitude. Covering that direction means deploying a second instance with the opposite configuration, which is a registry entry rather than a code change.

4.2 ReserveConservation

Where the sandwich rule reads sideways across one block, this one reads forward across many. It maintains a running balance sheet rebuilt from a proven, gap-checked event stream: lock, unlock, mint and burn events accumulated forward from an anchor height. The invariant is that minted must not exceed locked, and the shortfall when it does is the damage.

Only logs emitted by the custodian itself are counted, so a look-alike event from another contract in the same transaction is ignored. Transactions whose receipts show failure are skipped, because the protocol proves inclusion rather than outcome and a reverted lock locked nothing.

The reason this rule is a stream rather than a balance read is the protocol boundary named in Section 2.10: there are no state proofs, so there is no balance to read. That constraint produces the cursor, the ordered window, the optimistic settlement and the gap challenge, in that order.

4.3 ChainlinkFeed

Every dollar figure in the system has to come from somewhere, and a backend is not an acceptable answer for a protocol whose whole claim is that it does not take anyone's word for anything. Chainlink will never deploy to Creditcoin on this project's schedule, and it does not have to. Every feed update is an Ethereum transaction emitting an answer, so the Attestcoin Protocol can prove it and Creditcoin inherits the price permissionlessly.

This is a **source rule**, not a risk rule. It never pays anyone and can never be violated. Its only job is to advance a feed subject's accumulator with the answer Chainlink itself published, so that every other rule has a dollar figure that came off a chain.

Rules never take a price as an argument and never call an oracle. They resolve it through the core's attested-feed interface, which is the same read interface Watchtower exposes to any other Creditcoin contract. The system consumes its own primitive, which is a considerably better answer than passing a number around internally, and it means any Creditcoin contract can read a proven Ethereum price from Watchtower without asking permission.

POINT IT AT THE AGGREGATOR

The answer event is emitted by the aggregator, not by the consumer-facing proxy. A feed subject pointed at the proxy will never see an update, because the proxy emits nothing.

4.4 FailedTx

The smallest rule in the library and the one to read first. It is the single rule in the system that asserts a receipt status of zero rather than one, and that inversion lives in the rule rather than in the core, which is the whole reason the core can stay rule-agnostic. It reimburses the gas burned by a reverted transaction, capped by the policy.

Transaction types zero, one and two are decoded. Types three and four carry a fourth chunk whose split the protocol's decoder does not expose, so they are refused by name rather than reimbursed from a field the rule guessed at. Reimbursement for the fee-market transaction type uses the maximum fee per gas, an upper bound, because the encoding does not carry the effective price. Capped by the policy either way.

4.5 GapChallenge

Not insurance, but the system's own defence, described in Section 3.9. It is listed here because it is a rule identifier like any other, it is scoped by the same replay guard, and its evidence goes through the same entrypoint. The fact that it needs no special path is the strongest available evidence that the engine generalises.

4.6 Writing a fifth rule

Declare a window shape and a settlement mode, implement `evaluate` as a pure function of the verified window and the subject's accumulator, and register a subject bound to it. The engine, the vault, the proven feed, the bounty schedule and the challenge machinery all come for free.

The risks that fit this shape without any change to the engine include oracle deviation between a reported answer and a pool's realised price, liquidation fairness, censorship of a transaction across a run of blocks, bridge message replay, and real-world-asset attestation freshness. Each is a pure function of a window of proven Ethereum transactions, which is exactly the shape this engine adjudicates.

Rule identifiers are the hash of a versioned literal, computed identically in Solidity and in the shared package. Versioning lives in the string, so replacing a rule is a new identifier and a new subject rather than a silent change of meaning under existing cover.

Economic design

A proof system decides what is true. It does not decide who pays, who is paid for noticing, or what happens when a claimant is careless. Watchtower's vault is the only contract in the system that moves value, and the only contract permitted to move it is the core. A rule can compute a number; it can never reach the money.

5.1 Cover

A policy is bought against one subject for a dollar amount and a duration. The premium in version one is one percent of cover per thirty days, paid in the native token, quoted by reading the conversion rate from the vault and applying the contract's own formula, so the figure the interface shows is the figure the contract will demand.

Then nothing happens. The policyholder never files a claim, because the claim is the proof. If somebody proves a covered harm against that subject, the vault pays the beneficiary the rule named inside the same transaction that proved it. There is no adjuster, no submission form, no discretion and no waiting period.

5.2 Tranches and risk isolation

Staking to a subject's tranche is underwriting that specific risk. Premiums accrue as buyers arrive and payouts come out of the same pool. Risk is per subject rather than pooled across the protocol, so backing a Chainlink feed does not expose an underwriter to a bridge, and a catastrophic failure of one custodian cannot reach the capital behind an unrelated pool.

Withdrawals are blocked while a subject has any unresolved breach against it. Underwriters cannot exit between a proven violation and its settlement, which is the moment at which the incentive to leave is strongest. This is also why open breaches are counted rather than flagged, as described in Section 3.9.

5.3 Three bounds on a payout

Every settlement is bounded three ways at once, and the smallest wins.

BOUND	PROTECTS AGAINST
The holder's cover	Paying more than was insured. A policy is a ceiling, not an estimate.
The subject's staked tranche	Paying capital that nobody committed to this risk.
The registry's per-block payout cap	A wrong judgement by an untrusted rule, and grieving by repeated submission inside one block.

The third bound is the interesting one, because it is what converts an untrusted rule from a fatal risk into a priced one. A rule cannot move money and cannot write state, so the worst it can do is return an inflated number. The per-block cap puts a hard ceiling on what that costs, per subject, per block, and it is set by the registry rather than by the rule.

5.4 Premiums

Premiums accrue per unit of stake at the moment cover is bought, settled against a per-staker debt in the standard accumulator pattern, and are claimable without touching the stake itself. An underwriter earns from the policies written while they were backing the subject and nothing from the ones written before they staked.

A premium paid on a subject nobody is underwriting yet is parked rather than credited to whoever stakes next, because crediting it would pay somebody for risk they never carried. The individual stake ledger and the tranche total are deliberately different quantities: a payout reduces the tranche but not anybody's individual position, so the two diverge the moment a claim is paid, and premiums are shared over the positions that actually exist.

5.5 Bounties, priced off the protocol's own cost curve

Somebody has to notice. Watchtower pays whoever does, out of a per-subject bounty pool that anyone can top up. Funding a watch is how a protocol, a treasury or an individual says that a particular contract should be watched and puts money behind the statement.

The size of the bounty is where the economics and the Creditcoin infrastructure meet. Fresh evidence is cheaper to prove and more useful to everyone, so it should pay more. The usual way to express that is a clock, and a clock needs a trusted source of time and can be gamed at the margins. Watchtower does not use one. It reads freshness off the proof.

CONTINUITY ROOTS	MULTIPLIER	WHAT IT MEANS
at most 100	1.00x	Near a live attestation
at most 900	0.50x	Aging
above 900	0.20x	Past the twenty-four-hour checkpoint boundary

The prosecutor's incentive curve is the protocol's cost curve, so the two can never drift apart.

The number of continuity roots in a proof is exactly what the chain charges gas for, it is produced by the protocol, and a submitter cannot lower it without moving closer to a live attestation, which is the behaviour the schedule is trying to buy. No oracle, no timestamp, nothing to game.

5.6 Bonds and slashing

EVENT	BOND	CONDITION
Posted	0.1 CTC	Escrowed from the prosecutor when an optimistic breach opens
Returned	In full	The challenge window closed unchallenged and the claim settled
Slashed	To the challenger	A gap was proven inside the range the claim covered

A prosecutor who files an incomplete stream loses money to whoever notices. A prosecutor who files a complete one is paid a bounty. Neither outcome requires anybody to be trusted, which is the point.

5.7 Agents and their incentives

The cover buyer

Pays a premium for a ceiling on a specific, provable harm. Gains certainty of payout in exchange for accepting that only the enumerated risks are covered and only up to the amount insured. Has nothing to gain from lying, because there is nothing to tell anyone.

The underwriter

Earns premium income on one named risk, bounded by the tranche they staked and isolated from every other subject. Cannot exit during a dispute. Their downside is capped by the same per-block cap that bounds a rule error.

The prosecutor

Earns a bounty for supplying proof, weighted toward fresh evidence. Pays gas on every submission, including the ones that revert, and escrows a bond on every optimistic claim. Has no ability to forge a verdict, so the only profitable strategy available is to find real incidents quickly.

The challenger

Earns a prosecutor's bond for proving an omission. Needs only one skipped transaction and a single-transaction submission to collect, which makes careless prosecution unprofitable without requiring anyone to police it.

The watch funder

A protocol, treasury or individual who wants a contract observed. Pays into a bounty pool and thereby buys the attention of an open market of prosecutors, rather than hiring one monitoring service and trusting it.

Two properties of this set are worth naming. No agent is required to be honest for the system to produce correct verdicts, because verdicts are produced by verification rather than by assertion. And every agent who could harm the system is bonded, capped, or paying gas for the privilege, so misbehaviour costs money rather than merely failing.

Security model

Three components in this system could lie, and none of them can be believed. That is the whole security model, and it is the thing to read before anything else.

6.1 Trust boundaries

The prosecutor worker is untrusted

It assembles proofs and pays gas. It cannot forge a verdict, because the precompile verifies everything inside the settling transaction. A lying worker gets a revert and burns its own gas doing it. This is the property that allows the only long-lived process in the system to be run by anyone, on any host, with no permission.

Rules are untrusted

The core reaches `evaluate` by static call against a view function, so a rule cannot write state or move money. Its worst case, a wrong number, is bounded by the subject's per-block payout cap.

The frontend is untrusted

Every number that decides money is read from Creditcoin directly by the browser. The worker API supplies history, enrichment and narration only. If the two disagree, the chain wins.

The worker's store is a cache

Delete it and the system still works; only the charts go dark. There is no local cursor that could get ahead of the chain, because every scanner reads the proven cursor from Creditcoin on each tick and works forward from it. Progress is only ever recorded by a confirmed receipt.

The Proof Builder is untrusted

It produces proofs; the precompile decides whether they are valid.

What is left inside the trust boundary is Creditcoin's consensus, the Attestcoin Protocol's attestor set, and the Watchtower contracts themselves. The first two are the chain's own security assumptions, which any application on any chain inherits. The third is a fixed, published, tested body of code whose behaviour was decided before any incident occurred.

6.2 Adversaries and their bounds

ADVERSARY	ATTEMPT	OUTCOME
Dishonest prosecutor	Submit fabricated transactions or a fabricated ordering	The precompile refuses, or the contract re-derives the true indices and the shape check fails. Revert, gas lost, nothing paid.
Careless prosecutor	File a stream claim that skips a transaction	Any challenger proves the gap inside the window, the accumulator rolls back and the bond is slashed.
Replaying prosecutor	Resubmit the same evidence for a second payout	Rejected by the rule-scoped replay guard before a rule is consulted.
Hostile rule author	A rule that mutates state or moves funds	The static call reverts. A rule returning inflated damages is bounded by the per-block payout cap.
Exiting underwriter	Withdraw between a proven breach and settlement	Unstaking is blocked while any breach on that subject is unresolved.
Griefing submitter	Drain a tranche with repeated claims in one block	The per-block cap bounds total settlement per subject per block.
Compromised frontend	Display false balances or verdicts	Nothing that decides money is sourced from it. Chain reads are authoritative and independently checkable on the explorer.
Irrelevant challenge	Roll back a claim with an unrelated transaction in range	The challenge must carry a log emitted by the subject's own source contract.

6.3 Guarantees held up by tests

Seventy-nine tests run with no network access, alongside fuzzed invariants. Four are worth knowing by name, because each one holds up a claim made elsewhere in this paper rather than merely exercising a code path.

`A hostile rule cannot mutate state or drain the vault`

Deploys a rule matching the expected selectors but declaring `evaluate` state-mutating. The call reverts. This is the guarantee behind Section 3.3.

`The same transaction serves a stream and a challenge`

Fails outright under a global replay guard. This is why the guard is rule-scoped, and it was predicted before it was observed.

`The sandwich rule declines the unpriced side`

The rule refuses to price the side of a pair it was not configured for, rather than publishing a figure that is wrong by orders of magnitude.

`A real captured bundle replays offline`

A genuine proof bundle captured from the live testnet is replayed by the test suite, which asserts that the index derived from the real Merkle sibling path equals the one the Proof Builder reported. The project's founding thesis is committed as a passing assertion rather than left in a script's output. It skips loudly when no fixture is present, so a fresh clone with no API keys still runs green.

6.4 Verifying the Creditcoin integration itself

The precompile is native runtime code, so a forked development node does not have it, and unit tests substitute a stateless mock at the precompile address. A mock is a claim about how the real thing behaves, and a claim like that has to be checked against the live chain rather than against documentation. Three checks do it.

CHECK	WHAT IT ESTABLISHES
Precompile conformance	The live precompile at 0x0FD2 derives transaction indices with exactly the convention the mock implements, verified across indices zero through four thousand and ninety five.
Thesis check	For a given front-run, victim and back-run, the SDK, the precompile and an independent block explorer must all agree on the three positions.
Fixture capture and replay	A real bundle is frozen into the repository and replayed offline by the test suite, so the agreement is regression-tested rather than observed once.

This is the discipline the rest of the system depends on. Every claim in this paper about what the protocol does is either exercised by a test or measured from a receipt.

Implementation

7.1 Repository

contracts/	Foundry. The settlement contract, the rule library, the vault. 79 tests.
packages/shared/	Chain config, rule ids, types, generated ABIs.
packages/attestcoin/	Every conversation with the protocol: SDK client, proofs, pre-flight, gas.
apps/prosecutor/	The worker: three scanners, one submission path, indexer, REST and SSE.
apps/web/	Next.js. A landing page that makes the argument, then the application.
fixtures/	Proof bundles captured from the live testnet, replayed by the test suite.

Every conversation with the Attestcoin Protocol is isolated in one package, which is deliberate. An integration with a young protocol should be concentrated rather than scattered, so that tracking the protocol as it evolves is a change in one directory.

7.2 Contracts

CONTRACT	RESPONSIBILITY
WatchtowerCore	The settlement contract. The only contract that calls the precompile, and the only one that may move the vault.
SubjectRegistry	The catalogue: which risks exist, which rules may judge them, and what a mistake can cost.
UnderwritingVault	Cover, tranches, premiums, payouts, bounties, bonds and slashing.
The four rules	Pure judgement. Each is a view function the core reaches by static call.
Libraries	Window discipline and ordering, replay keys, damages arithmetic, price resolution.

The core deliberately does not inherit the protocol's reference base contract. That reference verifies a single transaction and dedupes globally on chain, height and index. Watchtower needs batch windows sharing one continuity proof, and rule-scoped dedupe, for the reasons given in Section 3.6. The reference contract is a good starting point and the wrong shape for this product, and saying so precisely is more useful than inheriting it and fighting it.

Deployment order is registry, vault, core, then rules. Rules read prices through the core's attested-feed interface, so they are deployed last, and there is no circularity because the core discovers rules through the registry at call time.

7.3 The prosecutor worker

The only long-lived process in the system, and completely untrusted. It finds candidate incidents, waits for attestation, builds proofs, pays gas and takes bounties.

SCANNER	WATCHES	CADENCE
Intra-block	Pools, for three consecutive indices with the same sender outside a stranger	12s
Stream	Custodians, for lock, unlock, mint and burn events	20s
Feed	A Chainlink aggregator, for published answers	120s

All three stay a fixed number of confirmations behind the source head, so evidence is finalised before it is proven. Candidates move through an explicit lifecycle, and the store is written atomically by writing to a temporary file and renaming, so a crash mid-write cannot corrupt it. Anything non-terminal is replayed on boot.

```

DETECTED → AWAITING_ATTESTATION → PROVING → PREFLIGHT → SUBMITTED
                                     ↳ CONFIRMED
                                     ↳ FAILED
                                     ↳ UNPROVABLE

```

Submissions are single-flight: one key, one in-flight transaction, one nonce, which is the simplest correct answer and removes an entire class of nonce-collision failures. The worker verifies the Creditcoin chain id and the protocol's chain-key mapping before its API binds, and refuses to start if either is wrong, so a misconfiguration becomes a readable boot failure rather than a confusing revert twenty minutes into a run.

Prosecution is also available over HTTP in two modes. Relayed, where the worker builds the proof, pays the gas and takes the bounty, so no wallet is required. Or self-signed, where the worker hands back ready-to-sign calldata and submits nothing, so the bounty goes to whoever signs. Neither can forge anything, because the settlement contract re-verifies every proof inside the settling transaction.

7.4 The dashboard

Every page renders with no wallet connected; only spending needs one. Reserves, cover, vault balances, the proven head and the proven price are read from Creditcoin directly by the browser. The worker API supplies history, candidate pipeline states, a prosecutor leaderboard and a server-sent event stream that narrates the attestation wait rather than hiding it behind a spinner.

The application is presented as a forensic document rather than a trading terminal: warm paper rather than near-black, hairline rules rather than shadows, one sage accent that means proven and is spent rarely, and monospace reserved for anything the chain produced, so proven data looks different from copy the authors wrote. Nothing on any screen blinks, pulses or breathes. The only motion is evidence arriving, once.

7.5 Engineering notes Creditcoin forced

Integrating with a young chain surfaces things no documentation would have predicted. These are recorded because they are the kind of knowledge that is expensive to rediscover.

Deployment runs on ethers rather than a Foundry script

Creditcoin's RPC omits a field from block headers that the Rust tooling requires to deserialise a block, so the scripted deployer fails. The dangerous part is how it fails: it broadcasts transactions it can then no longer track, which left a half-deployed system and three receipts out of nineteen transactions on the first attempt. A deployer built on ethers needs only raw transaction submission and receipt lookup, both of which the chain serves correctly, and the rewrite made both deployment steps idempotent.

The EVM version is pinned to a pre-merge specification

Post-merge specifications demand a header field this chain's headers do not carry, so the compiler target is pinned accordingly. This is the same root cause as the deployment issue, surfacing in a different tool.

Intermediate representation compilation is mandatory

Omitting it fails the build outright, because the verification path carries more live values than the legacy pipeline can allocate.

Gas is computed, never estimated

As described in Section 2.10, estimation against precompiles is unreliable, so every submission uses the SDK's own gas formula.

The SDK is the only supported path to the Proof Builder

The service's REST paths are not publicly documented, so hand-rolling HTTP against it is not an option. Provider instances are created once and shared, because attestation waiting polls and several scanners run concurrently.

Measured results

Every figure in this section came off a receipt from Creditcoin CC3 Testnet rather than out of an estimate or a documentation page. The maximum gas cap on Creditcoin is seventy-five million, so the share column is of one whole Creditcoin block.

8.1 Gas

SUBMISSION	CONTINUITY ROOTS	GAS USED	SHARE OF A BLOCK
Intra-block extraction, three transactions in one batch call, one shared continuity proof	1	1,233,005	1.64 %
Chainlink feed import, one transaction	4	519,540	0.69 %
Reserve conservation step, one transaction	50	502,390	0.67 %

A complete sandwich prosecution, three Ethereum transactions verified, three indices derived from the precompile, three receipts decoded, a rule evaluated and the vault moved, costs under two percent of one Creditcoin block.

8.2 A real mainnet sandwich, prosecuted

Ethereum Mainnet block 25,955,190, transaction indices 29, 30 and 31, on a Uniswap V2 DAI and WETH pool. A searcher bracketed a stranger's swap and took 0.001467 WETH out of their execution price.

Watchtower proved all three positions through the Block Prover Precompile in a single batch call, priced the damage at 3.77 US dollars using the ETH/USD answer Chainlink itself published, recorded the verdict on Creditcoin, and paid the prosecutor for the proof. The whole prosecution cost 1,233,005 gas.

Restitution was zero, and that is the system working rather than failing. The victim is a real stranger who never bought cover. The verdict stands on its own, on-chain, corroborated by the precompile's own verification event, and the incident record says plainly that nothing was paid rather than displaying a payout that did not happen. An insurance protocol that reported otherwise would be lying in the direction that flatters it.

25,955,190	29 · 30 · 31	\$3.77	1.64%
SOURCE BLOCK Ethereum Mainnet	INDICES PROVEN Derived by the precompile	DAMAGES At a proven Chainlink answer	OF ONE BLOCK 1,233,005 gas

8.3 What actually costs the gas

Continuity roots, and almost nothing else. The three-transaction sandwich, the most work the system ever does, used fewer continuity roots than the single-transaction reserve step, and the cost tracks the roots rather than the transactions.

This is the empirical basis for the bounty schedule in Section 5.5. The protocol's cost curve is already the right signal for freshness, so nothing else needed to be invented. It is also the reason batch verification matters so much: proving three transactions under one shared continuity proof is dramatically cheaper than proving them separately, because the shared part is the expensive part.

Deployment cost is negligible by comparison. The whole system deploys for a fraction of one Creditcoin token. The material spend in a demonstration is the capital seeded into the vault, and that is capital the operator owns inside the vault rather than a fee.

Limits

A protocol that pays out on proof has to be exact about what it has not proven. Every item below is stated here rather than left to be discovered, and each one is either a consequence of a protocol boundary or a version-one simplification with a known fix.

You cannot prove a negative on-chain

Sandwich claims are self-contained and settle instantly. Solvency claims depend on stream completeness, so they are bonded, challengeable and economically secured rather than self-evident. The asymmetry is a design feature, not an oversight.

Attestation takes about eight minutes end to end

Not the attestation cadence alone. The interface narrates the wait rather than hiding it, and nothing time-critical in the product depends on sub-minute proof latency.

No state proofs exist

The protocol proves transactions and receipts, not account state. Every balance in this system is therefore replayed from a proven event stream against an anchor, which is exactly why a stream can have a gap and why gaps are challengeable.

Damages are a documented model

The sandwich figure is the attacker's round-trip gain on the same pool in the same block, a lower bound that ignores the fee they paid. Gas reimbursement for fee-market transactions uses an upper bound on the gas price, because the encoding does not carry the effective price. The verdict is proven; the pricing is policy.

The sandwich rule prices one side of a pair

The deployed instance is configured for the eighteen-decimal side and declines the other rather than mispricing it. Covering both directions means a second instance.

Solvency payouts go to a subject's first cover buyer

The primary holder is first-come-first-served and the reserve rule returns no explicit beneficiary, so a second buyer on the same custodian would not be paid from a breach. Sandwich and failed-transaction verdicts name their beneficiary directly and are unaffected. Pro-rata settlement across policies is the fix.

Unstaking has no cooldown beyond the frozen check

An underwriter can exit right up until a breach is proven.

Cover is per subject, not per position size

A small policy pays at most its face value, regardless of how much the holder was actually trading.

Blob and delegation transaction types are refused

They carry a chunk whose split the protocol's decoder does not expose, so the failed-transaction rule refuses them by name rather than reimbursing from a field it guessed at.

The conversion from dollars to the native token is fixed in version one

Damages are priced in dollars because that is how the proven Chainlink answer is denominated, and settled in the native token at a configured rate. In production this becomes a token price feed, which is itself just another feed subject.

Registration is owner-gated

Subjects and rules are registered by the owner in version one. Because rules are pure and payouts are capped, opening this gate does not change the security model, only the permissioning.

Ethereum only, today

Every source chain the protocol adds becomes a new subject namespace at no cost to the engine. Nothing in the core is Ethereum-specific beyond the decoder and the rules.

Writability is in audit

Architected for, not demonstrated on. Until it ships, restitution is paid on Creditcoin rather than enforced at the source.

Roadmap

10.1 Source-chain enforcement through writability

The single largest change ahead is the outbound direction of the Attestcoin Protocol. Today a proven harm produces restitution on Creditcoin. With writability, a settled verdict publishes a message to the destination chain's inbox by way of an outbox, an attester set and a relayer, and the consequence is enforced where the harm occurred.

That changes the product category rather than merely extending it. Proving a custodian insolvent and paying its insured on another chain is insurance. Proving a custodian insolvent and halting its minting contract at the source is enforcement, and it is available to any protocol willing to delegate that authority to a proof rather than to a committee.

10.2 Permissionless rules and subjects

The registry gate is policy, not security. Rules are pure, payouts are capped per subject per block, and a hostile rule can already do no more than return a wrong number inside a bounded envelope. Opening registration turns the rule library into an open market, where a risk nobody has priced can be defined by whoever understands it, and capital decides whether it is worth underwriting.

10.3 Settlement and capital improvements

- Pro-rata settlement across all policies on a subject, which removes the first-buyer limitation.
- A proven native-token price feed replacing the fixed conversion rate, implemented as an ordinary feed subject.
- An unstaking cooldown, so that exiting before a breach is proven carries a delay rather than only a freeze afterwards.
- Position-aware cover, so that a policy can scale with the exposure it is protecting.

10.4 More risks, more chains

The rule shapes that fit today's engine without modification include oracle deviation, liquidation fairness, transaction censorship across a run of blocks, bridge message replay, and attestation freshness for real-world assets. Each additional source chain the protocol attests becomes a new subject namespace with no engine change, because nothing in the core is specific to Ethereum beyond the decoder and the rules that read its logs.

What this means for Creditcoin

Watchtower is an application, but the argument underneath it is about the chain it settles on. Creditcoin is not competing to host the same automated market makers as everyone else. It has a capability no other chain exposes, and that capability defines a category rather than a feature.

11.1 A settlement chain that adjudicates other chains

The conventional framing for a second chain is that it should attract the activity of the first. That is a difficult and crowded position. The framing this project argues for is different: a chain whose distinctive product is *judgement* about other chains, sold to those other chains and to their users.

Every claim of that kind has the same anatomy. A fact exists on a source chain, the fact matters financially, the source chain cannot act on it, and somebody currently gets paid to be trusted about it. Each such trusted party is a candidate for replacement by a Creditcoin contract that proves the fact instead, and each replacement is recurring demand for verification, for block space, and for the native token that pays for it.

11.2 The demand this creates

- **Block space with a floor under it.** Every prosecution is a Creditcoin transaction whose cost is dominated by protocol-native verification work, and that work exists only because the chain offers it.
- **Capital that stays.** Underwriting tranches, bounty pools and prosecutor bonds are all denominated in the native token and all locked against named risks rather than parked.
- **A reason for other chains to care.** An Ethereum protocol that wants its users covered against extraction does not have to deploy anything on Creditcoin. It funds a watch, and an open market of prosecutors does the rest.
- **A public good the chain can point at.** Watchtower exposes proven Ethereum prices and proven stream heads through a read interface any Creditcoin contract can consume, which makes verified foreign data a shared resource on the chain rather than a private one.

11.3 What Watchtower demonstrates about the protocol

A protocol is proven by the products that could not exist without it. This one exercises the Attestcoin Protocol across effectively its entire readability surface: batch and single verification, read-only verification, index derivation, the chain information precompile, the proof builder, the decoder, the gas helper, and the verification event. It does so from a testnet deployment reading Ethereum Mainnet, against real historical incidents, with measured costs.

The precompile is not a feature of Creditcoin. It is a different relationship between two chains, and insurance is only the first thing anyone thought to build on it.

Conclusion

Watchtower began from one observation about Creditcoin's infrastructure: the Block Prover Precompile will verify any Ethereum transaction, nothing binds that transaction to the contract asking, and the proof itself carries the transaction's position inside its block. Those three facts together let a Creditcoin contract prove statements about Ethereum's execution order.

Insurance is the product that makes the capability legible, because insurance is the industry where a trusted adjudicator has always been the cost of doing business. Remove the adjudicator and the product changes shape entirely: the claim is the proof, the policyholder files nothing, discretion disappears, and the payout is a consequence of verification rather than of a decision by anyone.

What exists today is a working engine with four rules, seventy-nine passing tests, fuzzed invariants, a real captured proof bundle replayed offline, a live deployment on Creditcoin CC3 Testnet reading Ethereum Mainnet and Sepolia, and a real mainnet sandwich prosecuted on-chain for under two percent of one Creditcoin block in gas. What is honestly not yet proven is listed in Section 9 rather than omitted.

The generalisation is the part worth keeping. One coordinate, supplied by the protocol rather than asserted by anybody, is enough to adjudicate a whole family of risks that the chains producing them cannot reach. Watchtower prices four of them. The engine does not care how many there are.

A Creditcoin contract can see what an Ethereum contract cannot: its neighbours. Everything in this paper is a consequence of that one sentence.

Addresses and networks

A.1 Networks

Network	Identifier	Role
Creditcoin CC3 Testnet	chain_id 102031	Settlement. Every verdict lives here.
Ethereum Mainnet	chainKey 3	Source. Real pools, real sandwiches, Chainlink aggregators.
Ethereum Sepolia	chainKey 1	Source. The staged custodian.

A.2 Attestcoin Protocol

SURFACE	ADDRESS OR VALUE
Block Prover Precompile	0x00FD2
ChainInfo Precompile	0x00FD3
Proof Builder	prover.cc3-testnet.creditcoin.network
Batch ceiling	10 transactions, 1,000 blocks
Maximum gas cap	75,000,000

REMEMBER

Code length at both precompile addresses is zero, and that is expected. They are native runtime code rather than EVM bytecode. Use the protocol library's presence check, never a code-size check.

A.3 Watchtower deployment

Contract	Address
WatchtowerCore	0x886498645c18a787f6283c25012771d77b068DF2
SubjectRegistry	0x063f5167Fe6F65c5B8c9F81862fE92678f248178
UnderwritingVault	0x5c9F0cF5D0057556B1A12D8f4028253c1f1C5A14
DemoBridge (Sepolia)	0x4044D34f8DF534B364B5EA337b72DD508198A5eF

A.4 The reference prosecution

FIELD	VALUE
Creditcoin transaction	0x9cad4c89ea95410e08896de3ae35d96a978348a9cc1a082824377f5505d29308
Source block	Ethereum Mainnet 25,955,190
Indices proven	29, 30, 31
Extraction	0.001467 WETH
Damages	3.77 USD at a proven Chainlink answer
Gas used	1,233,005 (1.64 % of one block)
Restitution	0 (the victim held no policy)

A.5 Live services

SERVICE	LOCATION
Dashboard and documentation	watchtower-attestation.vercel.app
Prosecutor worker health	watchtower-prosecutor.onrender.com/api/health
Creditcoin explorer	creditcoin-testnet.blockscout.com

Interfaces and data structures

A VERIFIED SOURCE-CHAIN TRANSACTION

```
struct VerifiedTx {
    uint64 chainKey;
    uint64 blockHeight;
    uint32 txIndex;      // produced by the precompile's calculateTxIndex, never by us
    address from;
    address to;
    bool success;        // receiptStatus == 1
    bytes encodedTx;     // rules decode logs from this with EvmV1Decoder
}
```

A RULE'S JUDGEMENT

```
struct Verdict {
    bool violated;
    address beneficiary; // zero ⇒ the vault pays the subject's primary cover holder
    uint256 damages;     // USD, 8 decimals
    bytes32 evidenceHash;
}
```

THE ACCUMULATOR A RULE READS, AND THE MUTATION IT REQUESTS

```
struct SubjectView {
    uint256 locked;
    uint256 minted;
    uint256 price; // 8 dp
    uint64 cursorHeight;
    uint32 cursorIndex;
}

struct AccDelta {
    int256 lockedDelta;
    int256 mintedDelta;
    uint256 price;      // 0 ⇒ leave unchanged
    uint64 newHeight;
    uint32 newIndex;
}
```

THE RULE INTERFACE, IN FULL

```
interface IConservationRule {
    function ruleId() external pure returns (bytes32);
    function windowShape() external pure returns (WindowShape);
    function settlement() external pure returns (Settlement);
    function name() external pure returns (string memory);

    function evaluate(bytes32 subjectId, VerifiedTx[] calldata window, SubjectView calldata state)
        external view returns (Verdict memory verdict, AccDelta memory delta);
}
```

THE READ INTERFACE ANY CREDITCOIN CONTRACT MAY CONSUME

```
interface IAttestedFeed {  
    function latestProvenHead(bytes32 subjectId) external view returns (uint64 height, uint32 index);  
    function reserves(bytes32 subjectId)          external view returns (uint256 locked, uint256 minted);  
    function price(bytes32 subjectId)              external view returns (uint256 answer, uint64 provenAtHeight);  
}
```

The feed interface is the system's public primitive. Watchtower consumes it internally, which is why a rule never takes a price as an argument, and any other Creditcoin contract can consume it to read a proven Ethereum price or a proven stream head without asking permission.

Rule identifiers and parameters

RULE IDENTIFIERS ARE THE KECCAK HASH OF THESE VERSIONED LITERALS

```
watchtower.rule.intra-block-extraction.v1
watchtower.rule.reserve-conservation.v1
watchtower.rule.chainlink-feed.v1
watchtower.rule.failed-tx.v1
watchtower.rule.gap-challenge.v1
```

PARAMETER	VERSION-ONE VALUE	SET BY
Premium rate	1 % of cover per 30 days	Vault
Base prosecutor bounty	0.05 CTC	Vault, owner-adjustable
Required bond	0.1 CTC	Vault, owner-adjustable
Fresh continuity threshold	100 roots	Vault constant
Stale continuity threshold	900 roots	Vault constant
Window ceiling	10 transactions	Protocol, enforced by the core
Block span ceiling	1,000 blocks	Protocol
Payout cap	Per subject, per block	Registry
Scan confirmations	8 blocks behind head	Worker

Appendix D · Glossary

Attestation

The commitment of a source-chain block digest to Creditcoin, after which transactions in that block become provable.

Chain key

The protocol's internal identifier for a source chain, deliberately not an EVM chain id.

Continuity proof

A chain of block digests linking the block under proof to an attested endpoint. Its length is the dominant gas cost and the system's measure of freshness.

Cursor

The coordinate of the last entry a subject's accumulator has applied. Every stream submission must begin strictly beyond it.

Evidence window

One or more source-chain transactions, each with a Merkle proof, sharing one continuity proof, verified in a single call.

Incident

The on-chain record produced by a judged window: subject, rule, coordinates, damages and beneficiary.

Prosecutor

Anyone who supplies proof of an incident and is paid a bounty for it. Untrusted by construction.

Rule

A pure view function that judges a verified window and returns a verdict and an accumulator delta.

Subject

One contract on one source chain bound to one rule. The unit that cover, capital, bounties and incidents all attach to.

Tranche

The capital staked against one subject's risk, isolated from every other subject.

Window shape

The structural constraint a rule declares in advance and the core enforces before judgement.

REFERENCES AND FURTHER READING

- Attestcoin Protocol documentation and the Creditcoin developer materials, for the Block Prover Precompile, the ChainInfo Precompile, the Proof Builder service and the attestation model.
- The protocol's Solidity contract package, for the native query verifier interface, the transaction decoder and the outbound writability suite.
- The Creditcoin universal smart contract SDK, for the proof builder client, the precompile clients and the gas helper.
- The Watchtower repository, for the contracts, the rule library, the prosecutor worker, the test suite and the captured proof fixtures.
- The Watchtower documentation site, which carries the operational detail this paper deliberately leaves out: environment, deployment order, hosting, scripts and troubleshooting.

Watchtower whitepaper, version 1.0, September 2026. Released under the MIT licence. Figures in this document were measured from receipts on Creditcoin CC3 Testnet and are reproducible by running the system described in Section 7.